

MIT 6.543.

Fall 2026

# Quantum Complexity Theory

Instructor: Anand Natarajan

9/10/26

# Lecture 1: Intro to BQP

- Complexity theory is about finding the mathematical limits of efficient computation

- Computers are physical objects, and efficiency is a physical notion (time, space, etc.)

So to understand complexity, need to use the right model of physics, quantum mechanics

Mission #1 of quantum complexity theory:

Understand the power & limitations of quantum computers

— Not only are computers physical, many computational problems arise from physics as well.

Even natural processes can be thought of as computations.

Mission #2 of quantum complexity theory:

Use computation as a lens to understand nature!

The basic method of complexity theory is to classify problems into complexity classes

typically defined by the computational resources needed to solve them

Recall  $P = \left\{ \begin{array}{l} \text{languages} \\ \text{decision problems} \end{array} \begin{array}{l} \text{solvable} \\ \text{classical} \end{array} \begin{array}{l} \text{by} \\ \text{algorithms} \end{array} \text{poly-time} \right\}$

$BPP = \left\{ \begin{array}{l} \text{languages} \\ \text{decision problems} \end{array} \begin{array}{l} \text{solvable} \\ \text{randomized} \end{array} \begin{array}{l} \text{by} \\ \text{classical} \end{array} \begin{array}{l} \text{algorithms} \\ \text{w/ bounded error} \end{array} \right\}$

if  $x \in L$ ,  $\Pr[M(x) = \text{YES}] \geq 2/3$   
if  $x \notin L$ ,  $\Pr[M(x) = \text{NO}] \leq 1/3$  } separated by a constant

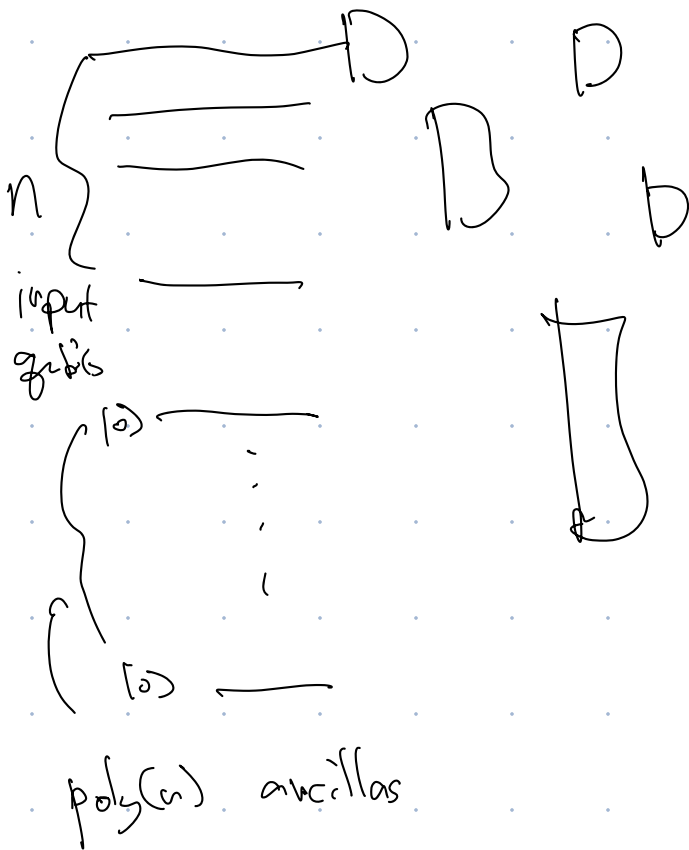
The star of quantum complexity theory is

$BQP = \left\{ \begin{array}{l} \text{languages solvable by a} \\ \text{poly-time b.e. quantum alg} \end{array} \right\}$

what does this mean?

Classically, poly-time defined using  
Turing machines.

Quantumly, easier to use circuits



poly(n) elementary gates

- No need to allow for intermediate measurements  
(just simulate coherently w/ ancilla)

- Can pick any reasonable universal set  
of elementary gates

(SK says you only pay a  $\text{polylog}(n)$   
cost to switch)

- Need some notion of **uniformity**: can't have  
a totally different circuit for each  $n$

"  $\exists$  classical TM  $M$  that runs in poly time  
and on input  $1^n$ , spits out  
a description of the circuit for inputs  
of length  $n$  "

Easy to see

$$P \subseteq BPP \subseteq BQP$$

↑  
by definition

↑  
simulate random coins using  
Hadamards

Also easy to see BQP is closed  
under complement (switching 'Yes' & 'No')

The thresholds  $2/3$ ,  $1/3$  are arbitrary  
and can be amplified

Just run the circuit many times and  
check that the fraction of accepts is above  
a threshold

Hoeffding if  $X_1, \dots, X_k$  are indep. r.v.  
in  $[0, 1]$

$$\Pr \left[ \left| \sum_{i=1}^k X_i - \mathbb{E} \sum_{i=1}^k X_i \right| \geq t \right] \leq 2e^{-\frac{2t^2}{k}}$$

So we can get  $1 - \exp(-k)$  vs  $\exp(-k)$  w/  $k$  repetitions

One nice property of  $P \subseteq BPP$  is that they "respect subroutines": a  $P$  machine given access to an oracle deciding another  $P$  language is equiv to a  $P$  machine without the oracle

The same is true for  $BQP$ , but proving it requires some care

How does a q-circuit access an oracle? for  $L$

$$U_L |x\rangle |b\rangle = \begin{cases} |x\rangle |b\rangle & \text{if } x \notin L \\ |x\rangle |b \oplus 1\rangle & \text{if } x \in L \end{cases}$$

unitary!

Suppose  $L$  is decided by a BQP machine.

The circuit gives some unitary  $U$

but it is not this one!

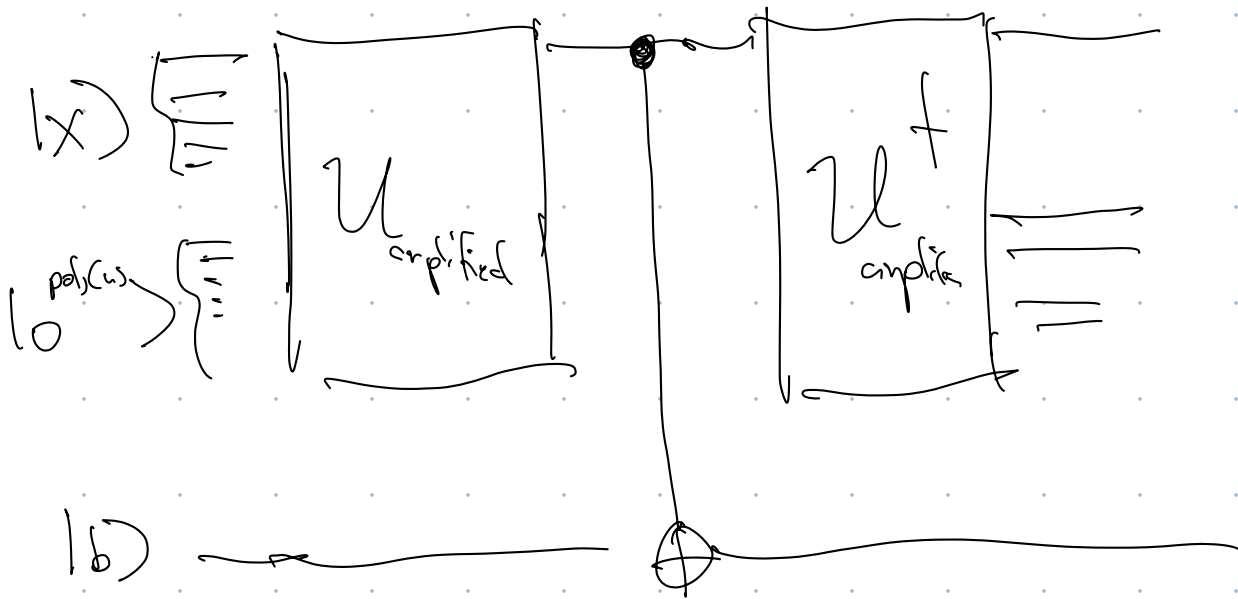
$$U|x\rangle|0^{\text{poly}(|x|)}\rangle = \alpha|0\rangle|\varphi_0\rangle + \beta|1\rangle|\varphi_1\rangle$$

$$|\beta|^2 \begin{cases} \geq 2/3 & \text{if } x \in L \\ \leq 1/3 & \text{if } x \notin L \end{cases}$$

can amplify to be  
exp close to 0 or 1

The real issue is the "junk"  $|\varphi_0\rangle, |\varphi_1\rangle$

Solution: "uncompute"



This implements  $U_L$  as desired

(except that we need ancilla, and there's some tiny error from amplification)

We have shown that

$$\text{BQP} = \text{BQP}$$

What about classical upper bounds  
on BQP?

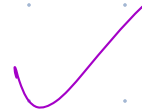
$$\text{BQP} \stackrel{??}{\leq} \text{BPP}$$

We hope not!!

$$\text{BQP} \stackrel{??}{\leq} \text{NP}$$

Probably not, stay  
tuned for evidence

$$\text{BQP} \leq \text{EXP}$$



Prove this w/ brute force simulation:

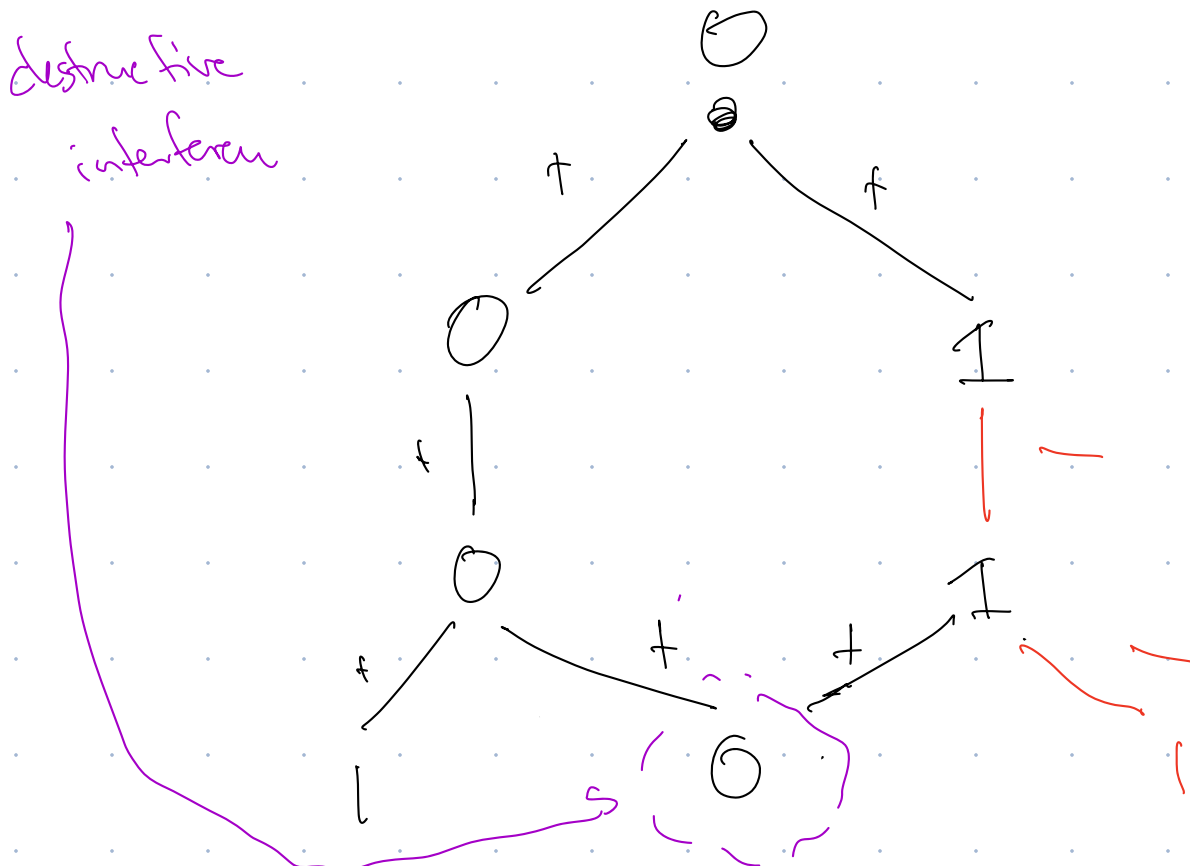
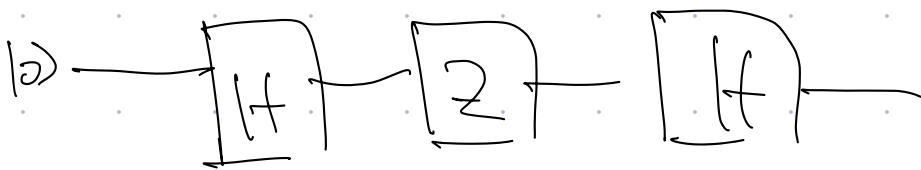
can calculate  $\left| \left( |0\rangle\langle 0| \otimes I \right) U |x\rangle |0^y\rangle \right|^2$   
in exp time

exp. big matrices  
and vectors

$$\text{BQP} \subseteq \text{PSPACE}$$

To show this, we need a new perspective:  
the "path integral"!

Easiest to fix  $\{H, CCZ\}$  as our universal set



In general, the state of the machine after a circuit with  $k$  Hadamard gates is  $\in \{\pm 1\}$

$$|\psi\rangle = \frac{1}{\sqrt{2^k}} \sum_{u \in \{0,1\}^n} \left( \sum_{\substack{\text{paths } p: \\ \text{end}(p)=u}} \text{sign}(p) \right) |u\rangle$$

can compute  
in poly time

$$\Pr[\text{accept}] = \frac{1}{2^k} \sum_{u \in \{0,1\}^{n-1}} \sum_{\substack{p, p': \\ \text{end}(p)=u, \text{end}(p')=u}} \text{sign}(p) \text{sign}(p')$$

This is a big sum so it can  
be computed in PSPACE

$$BQP \subseteq PP$$

PP is like BPP without bounded error

$$\begin{array}{ll} \text{if } x \in L, & \Pr[\text{accept}] \geq \frac{1}{2} \\ x \notin L, & \Pr[\text{accept}] < \frac{1}{2} \end{array} \quad \left. \vphantom{\begin{array}{l} \Pr[\text{accept}] \geq \frac{1}{2} \\ \Pr[\text{accept}] < \frac{1}{2} \end{array}} \right\} \text{no gap!}$$

To show this, we use the path integral!

$$\underbrace{\Pr[\text{accept}] - \Pr[\text{reject}]}_{\beta} = \frac{1}{2^k} \sum_{\substack{p, p': \\ \text{end}(p) = \text{end}(p')}} \text{sign}(p) \text{sign}(p') \quad \begin{array}{l} \text{end}(p) \text{ is } 1 \\ (-1) \\ \uparrow \\ \text{output bit} \end{array}$$

PP machine needs to guess whether

$\beta$  is  $+$  or  $-$ .

Idea: just sample a random term in the sum!

PP alg:

• Sample  $p, p'$  uniformly at random

• If  $\text{end}(p) \neq \text{end}(p')$ :

compute  $S = \text{sign}(p) \text{sign}(p') (-1)^{\text{end}(p)}$

if  $S = +1$ , output YES,  
else output NO.

Else:

output YES or NO uniformly at random.

$$\Pr[\text{accept}] = \Pr[\text{end}(p) \neq \text{end}(p')] + \Pr[\text{end}(p) = \text{end}(p')] \cdot \left( \frac{\text{prop-to-}}{\beta} \right)$$

Essentially this same calculation gives the tightest known classical upper bound on BQP

$$\text{BQP} \leq \text{AWPP}$$

ugly def, look it up!

9/15/26

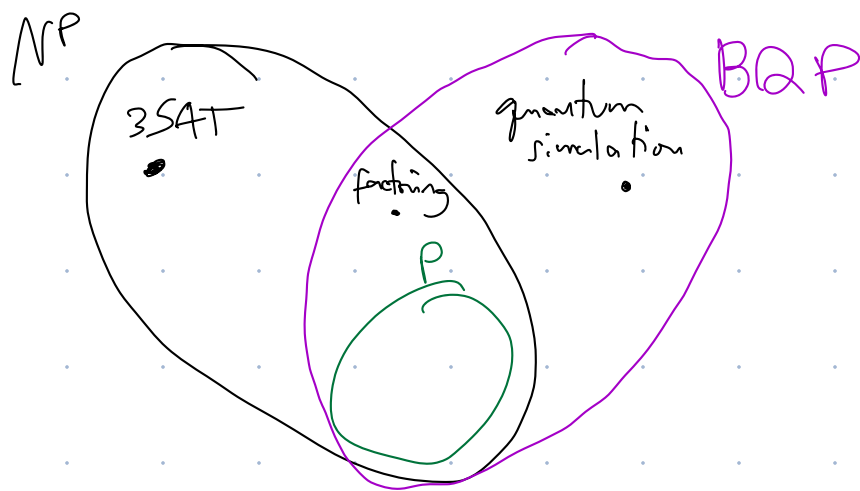
## Lecture 2: BQP and NP, and the BBBV lower bound for unstructured search

---

Understanding the relationship between BQP & NP  
is of great importance

- If  $BQP \stackrel{?}{\subseteq} NP$ , then every quantum computation has an efficiently checkable certificate
- If  $NP \stackrel{?}{\subseteq} BQP$ , then all NP-complete problems (e.g. 3SAT, 3 coloring, TSP) can be solved by quantum computers!

We currently believe that neither is true.



Today we'll focus on evidence that  $NP \neq BQP$ .

What we will show is an **oracle separation**:

$\exists$  oracle  $\mathcal{O}$  s.t.

$$NP^{\mathcal{O}} \neq BQP^{\mathcal{O}}$$

or unpacking this, there is some language

$L$  s.t.  $L \in NP^{\mathcal{O}}$  but  $L \notin BQP^{\mathcal{O}}$   
↑ related to  $\mathcal{O}$  ↑ provably

We can hope to prove this because deciding  $L$  requires some information about entries of  $\mathcal{O}$ .  
 So we will reduce the problem to the setting of "query complexity."

Note: An oracle separation does not imply that the classes are separated in reality!

$\exists$  oracles s.t.  $P^{\mathcal{O}} \neq NP^{\mathcal{O}}$  and also  
oracles s.t.  $P^{\mathcal{O}} = NP^{\mathcal{O}}$

What it means is that any proof that settles  $P$  vs  $NP$  (or  $BQP$  vs  $NP$ ) must be "non-relativizing": it must use techniques that break down in the presence of oracles.

Query complexity of search:

An oracle is a "magic box" that decides some language in unit cost.

Given input  $x \in \{0,1\}^*$ ,  $\mathcal{O}$  tells you  
YES or NO

Alternate perspective. The truth table of

$\mathcal{O}$  is a computational input that  
can only be accessed by querying  $\mathcal{O}$   
on inputs  $u \in \{0, 1\}^k$   
 $\uparrow$   
index into truth table

This motivates the model of query complexity

A "Query P" machine, given access to some  
input  $x \in \{0, 1\}^{\tilde{N}}$  should decide if  $x \in S_{\text{YES}}^{(n)}$   
or  $x \in S_{\text{NO}}^{(n)}$   
 $N := 2^m$

using  $\text{poly}(n)$  queries to  $x$   $\leftarrow$  only count queries, not runtime!

A "Query BQP" machine is the same, but  
gets to make quantum queries

= apply  $U_x : U_x |u\rangle |b\rangle \mapsto |u\rangle |b \oplus x_u\rangle$

A "Query NP" machine also gets a  
witness  $w \in \{0,1\}^{\text{poly}(n)}$

- If  $x \in S_{\text{YES}}^{(n)}$ ,  $\exists w$  s.t.

$M(n)$  outputs YES

- If  $x \in S_{\text{NO}}^{(n)}$ ,  $\forall w$ ,  $M(n)$  outputs NO

It turns out that to find an  
oracle separation between BQP & NP  
it will be enough to find a

query promise problem specified

by sets  $S_{\text{YES}}^{(n)}$  &  $S_{\text{NO}}^{(n)}$  for

each  $n$ , s.t. a Query NP machine can  
solve it but a Query BQP machine cannot

Aside: a "promise problem" is a decision problem where the input is promised to come from some set. E.g. here, we only care about  $x \in S_{\text{YES}}^{(n)} \cup S_{\text{NO}}^{(n)}$ , not arbitrary  $x \in \{0,1\}^n$ .

This "query separation" can be lifted by very standard diagonalization arguments to an oracle separation  $\text{NP}^O \not\subseteq \text{BQP}^O$ , so we'll focus on the query setting.

# A candidate problem: Unstructured Search

NP is all about "finding a needle in a haystack." Think about SAT: want to find an input that makes the formula evaluate to 1, not 0.

Inspired by this, define the following problem

$$S_{\text{YES}}^{(n)} = \{ \text{all } x \in \{0,1\}^{2^n} \text{ except } 0^{2^n} \}$$

$$S_{\text{NO}}^{(n)} = \{ 0^{2^n} \}$$

Basically, want to decide if  $\exists u \in \{0,1\}^n$

$$\text{s.t. } x_u = 1$$

↖ analogous to evaluating a SAT formula  $\Phi$  at  $u$ .

## NP containment

Kind of obvious: a good witness  
is some  $w$  s.t.  $X_w = \mathbb{I}$ !

## P lower bound:

Fix some **Query P** machine  $M$ .

If  $M$  solves Unstructured Search, then

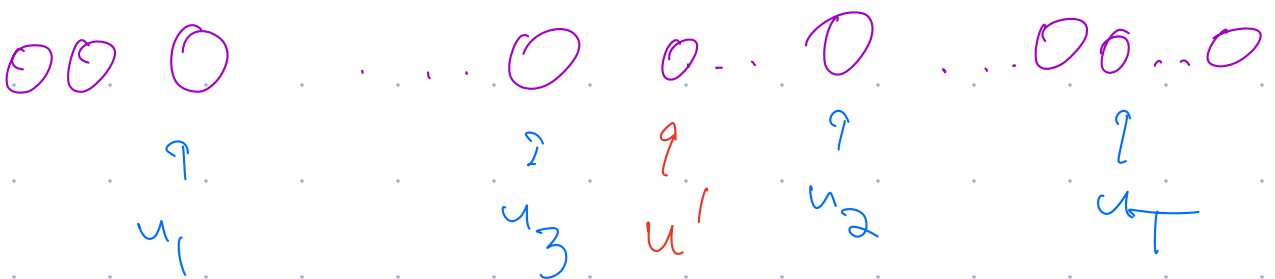
$M$  must reject  $X = 0^{2^n}$ .

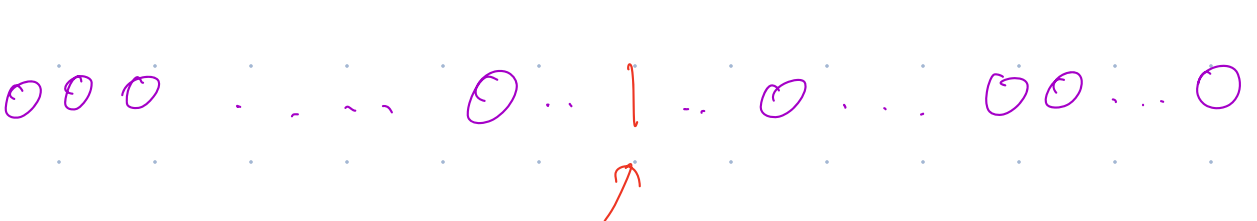
Let  $u_1, u_2, \dots, u_T$

total  $\nearrow$  # of queries

be the positions that  $M$  queries on

input  $0^{2^n}$ . Since  $T$  is  $\text{poly}(n)$ , it must be less than  $2^n$ , so there is some location  $u'$  that has not been queried.

$x =$    $000 \dots 00 \dots 00 \dots 0$   
 $\uparrow \quad \uparrow \quad \uparrow \quad \uparrow \quad \uparrow$   
 $u_1 \quad u_3 \quad u' \quad u_2 \quad u_4$

$x' =$    $000 \dots 0 \dots 1 \dots 0 \dots 00 \dots 0$   
 $\uparrow$   
 flip the value at  $u'$

Now,  $x' \in S_{\text{yes}}$ , but  $M$  must still  
 output No on  $x'$ , because  $x \neq x'$  look

identical to  $M$ !

Therefore,  $M$  gets  $x'$  wrong. ~~scribble~~

In fact, this argument applies to any deterministic machine that makes fewer than  $2^n$  queries. Can even extend to randomized machines (see Pset 1!)

BQP lower bound:

None of the ideas from the P case seem to work!

Problem 1: quantum algo. query in

superposition. Even 1 query can

"see" the entire input  $x$

$$\frac{1}{\sqrt{2^n}} \sum_{u \in \{0,1\}^n} |u\rangle |0\rangle \xrightarrow{U_x} \frac{1}{\sqrt{2^n}} \sum_{u \in \{0,1\}^n} |u\rangle |x_u\rangle$$

We know this state can sometimes reveal a lot of information about  $x$ ! E.g. the Bernstein Vazirani alg uses this to learn an entire linear function in one query.

Problem 2: specifically for unstructured search, Grover showed we can do way better than  $\Theta(2^n)$  queries.

The # of queries needed is

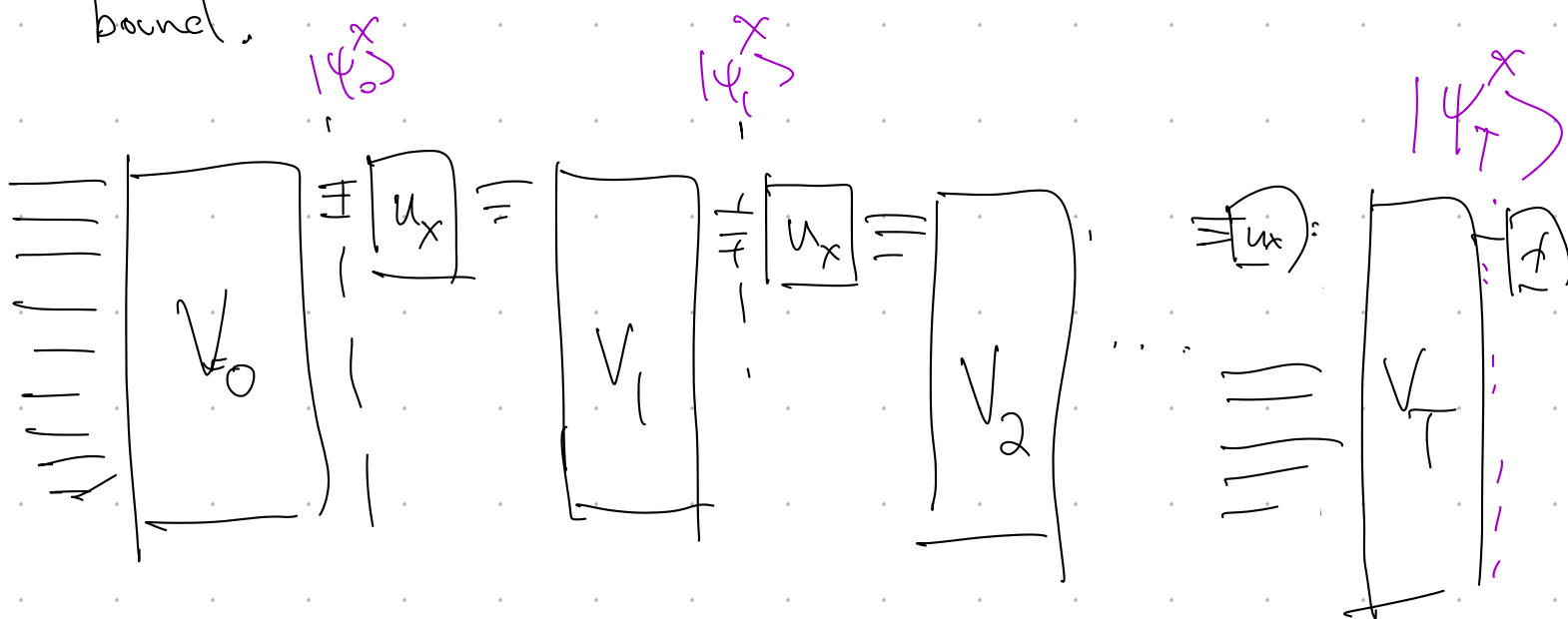
$$O(\sqrt{2^n})$$

It turns out we can get around these problems and show that Grover's alg. is essentially optimal, by some more careful accounting.

$T_1$  is the

B B B  $\checkmark$  '94 ← older than Grover's alg!  
 emmett erstein rassord azarani

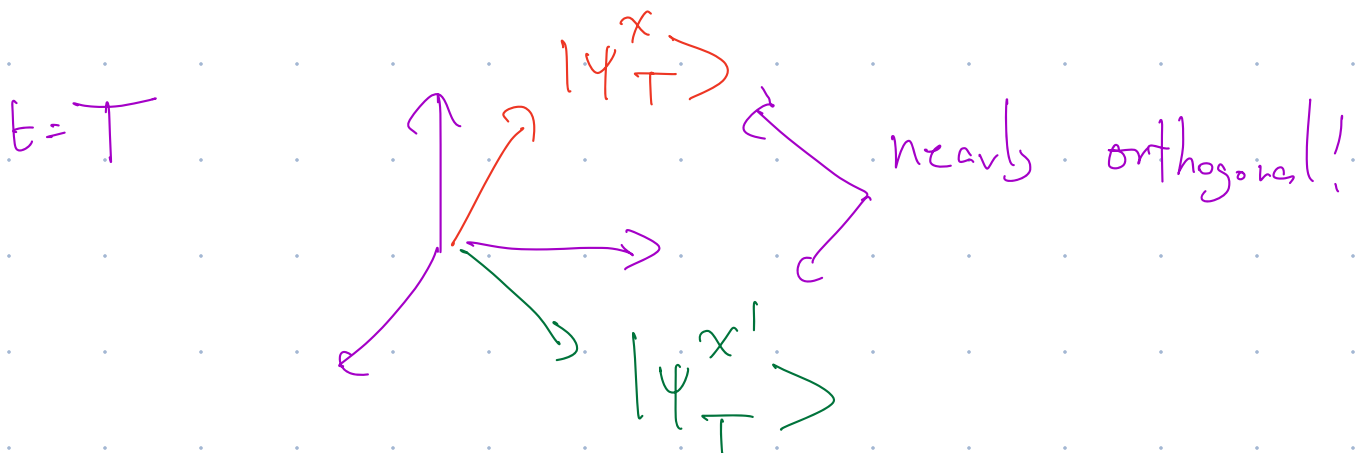
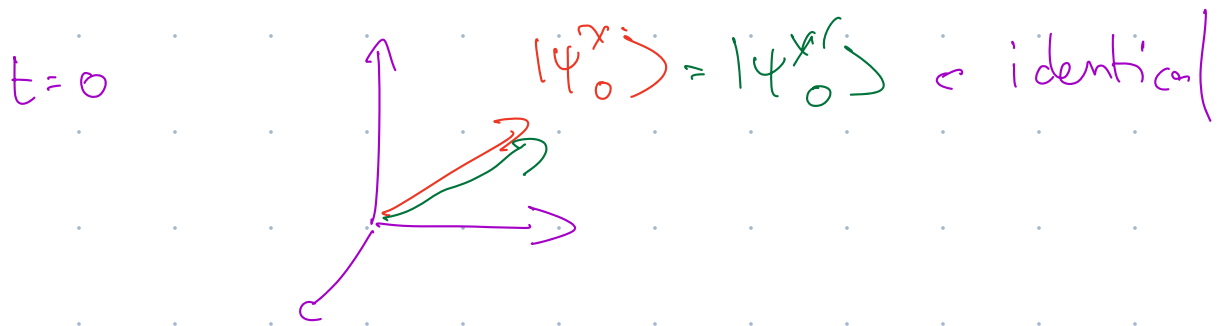
bound,



↑  
 A general circuit making  $T$  queries.

Idea: For different  $x, x'$ , track how far apart  $|\psi_t^x\rangle$  and  $|\psi_t^{x'}\rangle$  are

If  $x \in S_{No}$ ,  $x' \in S_{Yes}$ , need



If a quantum alg. can pull apart  
the two states by only a little bit in  
Euclidean distance with each query, then  
 $T$  must be large for any successful  
quantum alg.

More precisely:

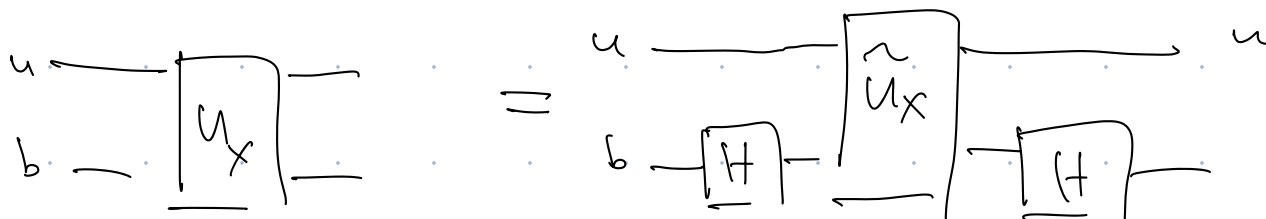
First, a slight change to the oracle:

instead of

$$U_x |u\rangle |b\rangle = |u\rangle |b+x_u\rangle, \text{ work with}$$

$$\tilde{U}_x |u\rangle |b\rangle = (-1)^{x_u \cdot b} |u\rangle |b\rangle$$

This is totally equivalent



$$\text{Let } |\psi\rangle = \sum_{u,b} \alpha_{u,b} |u\rangle |b\rangle |\varphi_{u,b}\rangle$$

oracle acts here only

We want to compute

$$\| \tilde{U}_x |\psi\rangle - \tilde{U}_{x'} |\psi\rangle \|_2$$

$$= \left\| \sum_{u,b} \alpha_{u,b} |u\rangle \left( \binom{b x_u}{(-1)} - \binom{b x'_u}{(-1)} \right) |b\rangle |\varphi_{u,b}\rangle \right\|_2$$

$$= \left\| \sum_{u: x_u \neq x'_u} \pm 2 \alpha_{u,1} |u\rangle |1\rangle |\varphi_{u,1}\rangle \right\|_2$$

$$= 2 \sqrt{\sum_{u: x_u \neq x'_u} |\alpha_{u,1}|^2}$$

So to pull the states apart, the alg.  
must have high "superposition weight"  
on positions where  $x \neq x'$  differ.

We can now show our lower bound  
by a hybrid argument + induction

$$\Delta_t^{x, x'} =$$

$$\| |\psi_t^x\rangle - |\psi_t^{x'}\rangle \|_2 = \| V_t \tilde{u}_x |\psi_{t-1}^x\rangle - V_t \tilde{u}_{x'} |\psi_{t-1}^{x'}\rangle \|_2$$

$$= \| \tilde{u}_x |\psi_{t-1}^x\rangle - \tilde{u}_{x'} |\psi_{t-1}^{x'}\rangle \|_2$$

$$= \| \tilde{u}_x |\psi_{t-1}^x\rangle - \tilde{u}_{x'} |\psi_{t-1}^x\rangle + \tilde{u}_{x'} |\psi_{t-1}^x\rangle - \tilde{u}_{x'} |\psi_{t-1}^{x'}\rangle \|_2$$

$$\leq \| (\tilde{u}_x - \tilde{u}_{x'}) |\psi_{t-1}^x\rangle \|_2 \leftarrow \eta_t^{x, x'}$$

↑

triangle  
ineq.

$$+ \| \tilde{u}_{x'} (|\psi_{t-1}^x\rangle - |\psi_{t-1}^{x'}\rangle) \|_2$$

$$\Delta_{t-1}^{x, x'}$$

Claim: Let  $x = 0^N$ , & let  $n$  be the uniform distribution over weight-1 strings in  $\{0, 1\}^N$ . Then

$$\mathbb{E}_{x' \sim n} \eta_{t, x, x'} \leq \frac{2}{\sqrt{N}} \quad \forall t$$

Pf: Let  $|\psi_{t-1}^x\rangle = \sum \alpha_{u,b} |u\rangle |b\rangle |\psi_{u,b}\rangle$

By our calculation above,

$$\mathbb{E}_{x' \sim n} \eta_{t, x, x'} = \sum_{u \in [N]} 2 |\alpha_{u, 1}|$$

↑  
index where  
 $x'_u = 1$

$$= \frac{2 \sum_{u \in [N]} |\alpha_{u,i}|}{N}$$

$$= \frac{2}{N} \langle \vec{\alpha}, \vec{\text{sgn}(\alpha)} \rangle$$

$$\begin{pmatrix} \alpha_{0,i} \\ \alpha_{1,i} \\ \alpha_{2,i} \\ \vdots \\ \alpha_{N,i} \end{pmatrix}$$

$$\begin{pmatrix} \text{sgn}(\alpha_{0,i}) \\ \text{sgn}(\alpha_{1,i}) \\ \vdots \\ \text{sgn}(\alpha_{N,i}) \end{pmatrix}$$

entries are  $\pm 1$

Cauchy Schwarz

$$\leq \frac{2}{N} \cdot \|\vec{\alpha}\|_2 \cdot \|\vec{\text{sgn}(\alpha)}\|_2$$

normalization of state

$$\leq \frac{2}{N} \cdot 1 \cdot \sqrt{N} = \frac{2}{\sqrt{N}}$$

Now, putting it all together, we have

$$\mathbb{E}_{x'_N} \Delta_T^{O^N, x'} \leq \mathbb{E}_{x'_N} \left( \eta_T^{O^N, x'} + \Delta_{T-1}^{O^N, x'} \right)$$

$$\leq \frac{2}{\sqrt{N}} + \mathbb{E}_{x'_N} \Delta_{T-1}^{O^N, x'}$$

$$\leq \frac{2}{\sqrt{N}} + \frac{2}{\sqrt{N}} + \mathbb{E}_{x'_N} \Delta_{T-2}^{O^N, x'}$$

$$\leq \frac{2T}{\sqrt{N}} + \underbrace{\mathbb{E}_{x'_N} \Delta_0^{O^N, x'}}_{=0}$$

$\Rightarrow$  Any quantum alg. that solves  
unstructured search w/ bounded error needs  
 $T = \Omega(\sqrt{N})$  queries!



9/17/26

# Lecture 3A: The Polynomial Method

The BBBV method is hard to directly apply to most problems. Sometimes alternate perspectives get us much farther by exploiting symmetry in the problem.

A very powerful one is the polynomial method, which views an algorithm as a low-degree polynomial.

$$\equiv \boxed{V_0} = \boxed{\Theta} = \boxed{V_1} = \boxed{\Theta} = \boxed{V_2} \dots$$

Lemma: After  $t$  steps,

$$\text{the state } |\psi_t\rangle = \sum \alpha_i |i\rangle$$

↑  
degree- $t$  poly in  
the bits

$x_1 \dots x_N$  of  
the oracle

Pf: By induction!

Base case:  $t=0$ , obvious

$t \rightarrow t+1$

$$\Theta |\psi_t\rangle = \Theta \sum_{i,b,z} \alpha_{i,b,z} |i\rangle |b\rangle |z\rangle$$

$$= \sum_{i,b,z} (-1)^{b \cdot x_i} \alpha_{i,b,z} |i\rangle |b\rangle |z\rangle$$

$$= \sum_{i, b, z} \underbrace{(1 - 2bx_i)}_{\text{degree 1}} \underbrace{\alpha_{i, b, z}}_{\text{degree } t} \quad |i| > |b| > |z|$$

degree  $t+1$  polynomial!

□

Cor:  $\Pr[\text{acceptance}^X] =$  degree  $2T$  poly in  
of  $T$ -query alg.  $x_1 \dots x_N$

The 2 comes from squaring!

So if you can show that no low-degree poly matches the promise problem you want to decide, you show that there's no alg.!

But it's very hard to reason about multivariate polynomials with lots of variables. So often one uses symmetry of some kind to reduce the number of variables.

# The unstructured search problem:

Idea: consider distributions on inputs that are symmetric under permutations of  $x$ .

$\mu_r :=$  uniform dist. over  $x$  s.t.  $|x| = r$   
↑  
Hamming weight

Claim: If  $p(x_1, \dots, x_N)$  is degree  $d$  multivariate,

$g(r) := \mathbb{E}_{x \sim \mu_r} p(x_1, \dots, x_N)$  is degree  $d$  poly in  $r$ .

$$\text{Pf: } \mathbb{E}_{x \sim \mu_r} x_1 x_3 x_{17} = \mathbb{P}_r \left[ \begin{array}{l} \text{all 3 of } x_1, x_3, \\ \text{and } x_{17} \text{ are equal to } 1 \end{array} \right]$$

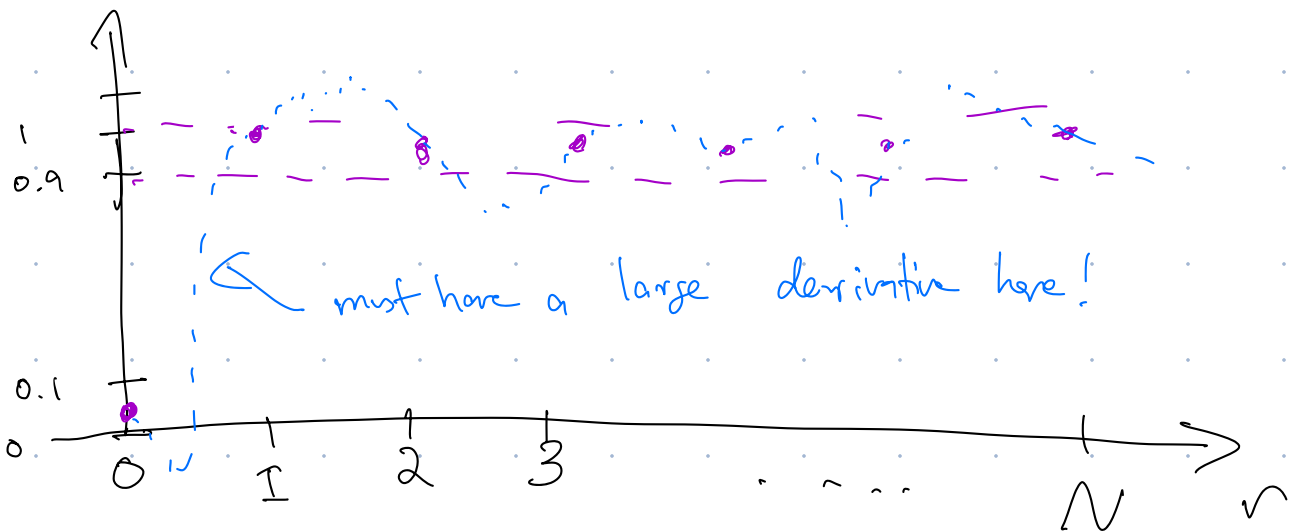
$$\begin{aligned} &= \frac{\binom{N-3}{r-3}}{\binom{N}{r}} \\ &= \frac{(N-3)!}{(r-3)! (N-r)!} \\ &= \frac{N!}{r! (N-r)!} \end{aligned}$$

$$= \frac{r(r-1)(r-2)}{N(N-1)(N-2)}$$

degree 3  
poly in  
 $r$ !

Easily generalizes to monomials of any degree.

Now, if we have an alg that solves  
unstructured search in  $T$  queries  $\Rightarrow$  a degree  $-2T$   
polynomial  $q$  that looks like this



Note: no constraint on  $q$  outside of integer  
points (need not even be positive).

It turns out no low-degree poly can look  
like this!

Markov's other inequality:

$$\max_{-1 \leq x \leq 1} \left| \frac{dp(x)}{dx} \right| \leq d^2 \max_{-1 \leq x \leq 1} |p(x)|$$

↗  
degree(p)

This immediately gives us a useful bound if  $q$  stays bounded at non-integer points.

If  $q$  gets large at non-integer points, then you get a large derivative in between integer points and you can still apply this.

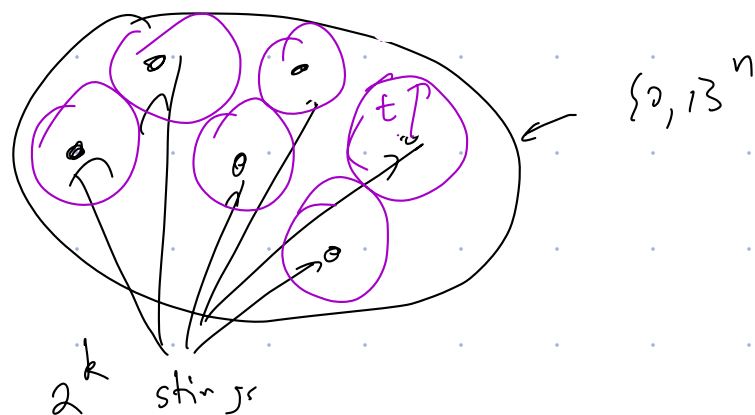
Upshot: Need  $\deg(q) \geq \Omega(\sqrt{N})$ .

Getting the right symmetrization is a bit of an art!

7/22/26

## Lecture 4: Error Correction Bootcamp

A classical code is a collection of bit strings

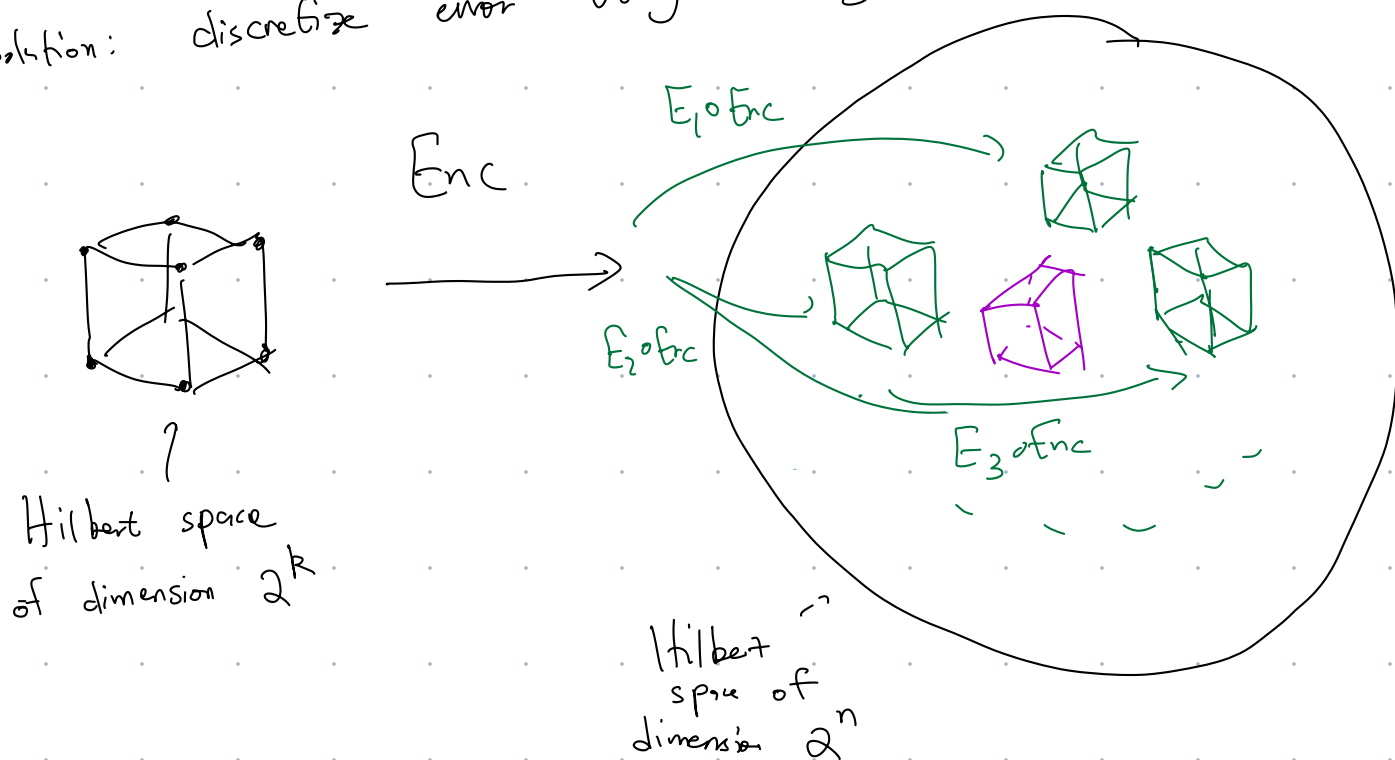


Can correct any error on at most  $t$  locations

### Quantum codes:

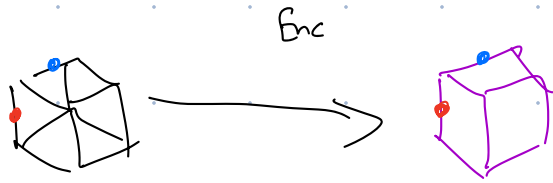
Classical errors are discrete, but quantum errors are continuous

Solution: discretize error using linearity of Q.M.



For now, think of  $E_1, E_2, \dots$  as a basis for the set of errors we want to correct.

In order to recover information:



has to preserve the structure of the subspace

$$\langle a | b \rangle = 0 \iff \langle \bar{a} | \bar{b} \rangle = 0$$

$\text{"(Enc}(\cdot\text{)}\text{)}^\dagger \quad \text{"Enc}(b)\text{"}$

Similarly, for each error  $E_i, E_j$

$$\langle a | b \rangle = 0 \implies \langle \bar{a} | E_i^\dagger E_j | \bar{b} \rangle = 0$$

More generally, what does it mean to correct an error?

$$\exists \mathcal{U}_{\text{rec}} \text{ s.t. } \mathcal{U}_{\text{rec}} E_i |\bar{\psi}\rangle = |\psi\rangle \otimes | \text{junk}_i \rangle$$

$\uparrow$   
 "recovery map"

$\frac{1}{\|E_i|\bar{\psi}\rangle\|} |\text{syndrome}_i\rangle$

By linearity, this implies that for all  $|\psi\rangle$ ,

$\|E_i|\bar{\psi}\rangle\|$  is the same value  $\sqrt{p_{E_i}}$  depending only on  $i$ .

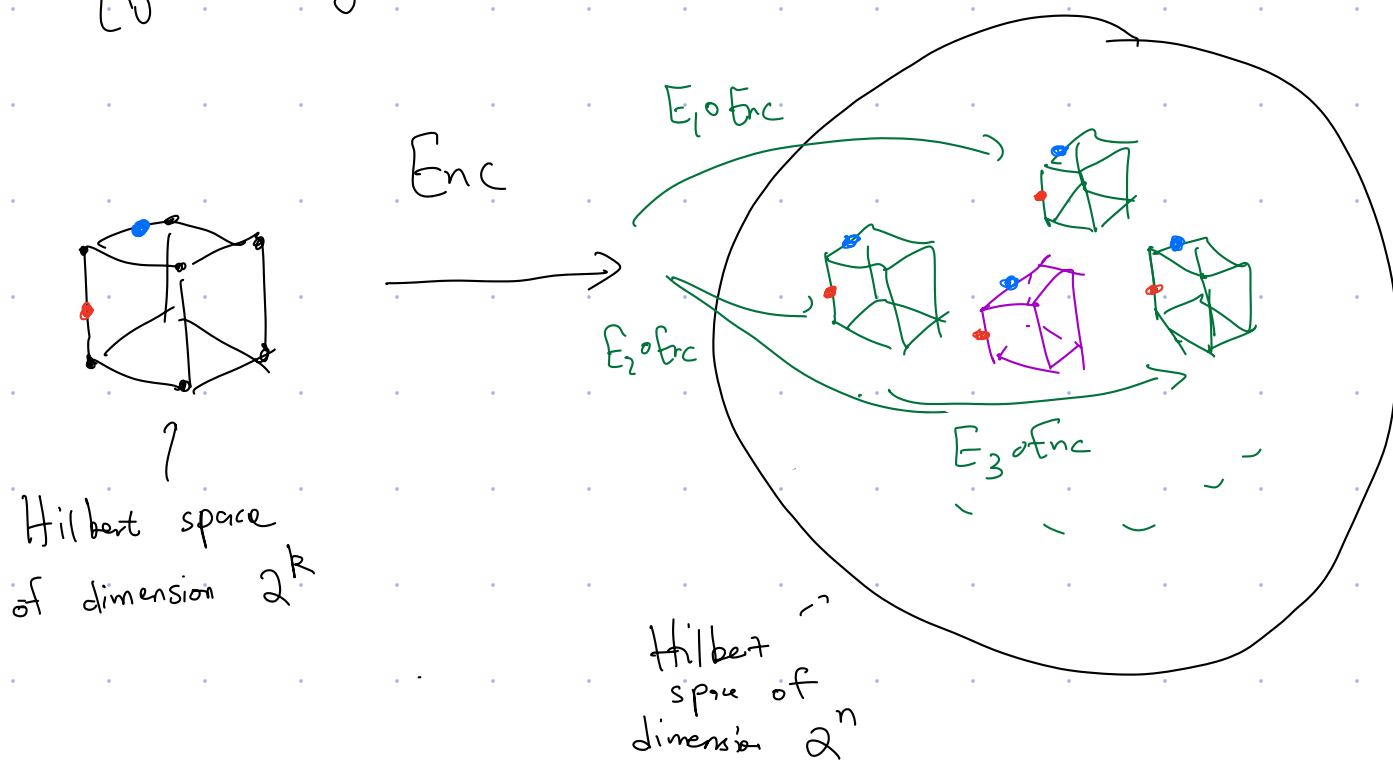
Now we can derive some more conditions:

$$\begin{aligned}
 \langle \bar{a} | E_i^\dagger E_j | \bar{b} \rangle &= \langle \bar{a} | E_i^\dagger U_{\text{rec}}^\dagger U_{\text{rec}} E_j | \bar{b} \rangle \\
 &= \langle \bar{a} | \bar{b} \rangle \cdot \frac{\langle \text{syndrome}_i | \text{syndrome}_j \rangle}{\sqrt{P_{E_i} \cdot P_{E_j}}} \\
 &= \langle \bar{a} | \bar{b} \rangle \cdot \underbrace{\eta_{ij}}_{\text{independent of } |\bar{a}\rangle, |\bar{b}\rangle} \quad (*)
 \end{aligned}$$

Equation (\*) is called the **Knill-Laflamme** condition and turns out to be sufficient for  $U_{\text{rec}}$  to exist! (We won't prove this here)

A useful special case for intuition is when

$$\eta_{ij} = \delta_{ij}.$$



In this special case, each "box" in the encoded space is an orthogonal "copy" of the original  $2^k$ -dimensional Hilbert space.

$U_{\text{rec}}$  operates by "extracting" the information of which box the state is in, without disturbing the state itself.

Pauli Errors : Typically, we take

$\{E_1, E_2, \dots\}$  to be the set of all Pauli operators supported on up to weight  $t$ . In this case, we say the code corrects "all weight  $t$  errors"

Classical vs. Quantum Codes:

Intuition: classical codes operate by copying information.

E.g. the repetition code

$$0 \rightarrow \overline{0} = 000 \dots 0$$

$$1 \rightarrow \overline{1} = 111 \dots 1$$

Quantum codes cannot work like this, b/c of the no-cloning thm.

Indeed, quantum codes satisfy the **local indistinguishability** property: if a code corrects  $t$  errors, then all code states look the same on regions of size  $\leq t$ .

Pf. Let  $|a\rangle, |b\rangle$  be two states  
 $\text{Enc} \downarrow$   
 $|a\rangle, |b\rangle$

To show that  $|a\rangle, |b\rangle$  are locally indistinguishable, let  $M$  be a measurement operator supported on a region  $T$  of  $\leq t$  qubits. Then we must show that  $\langle a | M | a \rangle = \langle b | M | b \rangle$

$$\begin{aligned}
 \text{Indeed, } \langle a | M | a \rangle &= \langle a | \sum_i m_i E_i | a \rangle \\
 &= \sum_i m_i \langle a | E_i | a \rangle \\
 &= \sum_i m_i \langle a | a \rangle \cdot \eta_{0i} \quad \leftarrow \text{KL} \\
 &= \sum_i m_i \langle b | b \rangle \cdot \eta_{0i} \\
 &= \sum_i m_i \langle b | E_i | b \rangle \\
 &= \langle b | M | b \rangle
 \end{aligned}$$

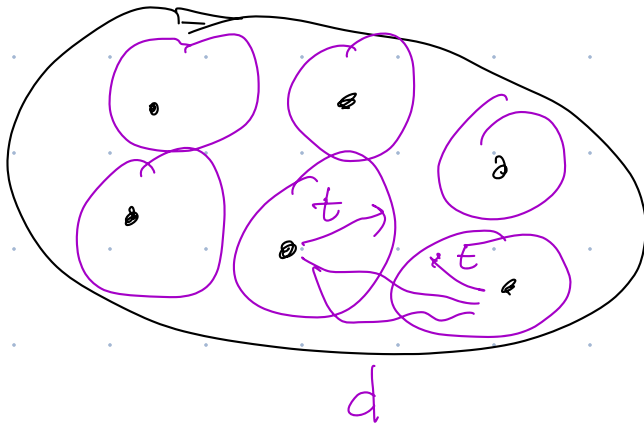
Intuition: quantum codes "stretch out" information using entanglement 

## Constructions: linear & stabilizer codes

Classically: a linear code is a subspace of  $\mathbb{F}_2^n$  of dimension  $k \leftarrow 2^k$  vectors

Distance := min Hamming weight of a nonzero codeword  
= min distance between codewords  
(by linearity, difference between codewords is itself a codeword)

A code with distance  $d$  can correct  $\left\lfloor \frac{d-1}{2} \right\rfloor$  errors



Notation:  $[n, k, d]$  code

Examples: 1) Repetition code  $[n, 1, n]$

span  $\{00 \dots 0, 11 \dots 1\}$

2) Reed-Solomon code

Details are a little complex  
but here's the intuition.

Let  $\mathbb{F}_q$  be a finite field of order  $q$

$|\mathbb{F}_q| = q \leftarrow$  represent each element with  $\log q$  bits

A finite field is a finite set that supports addition, multiplication, & division. So we can think of polynomials over it.

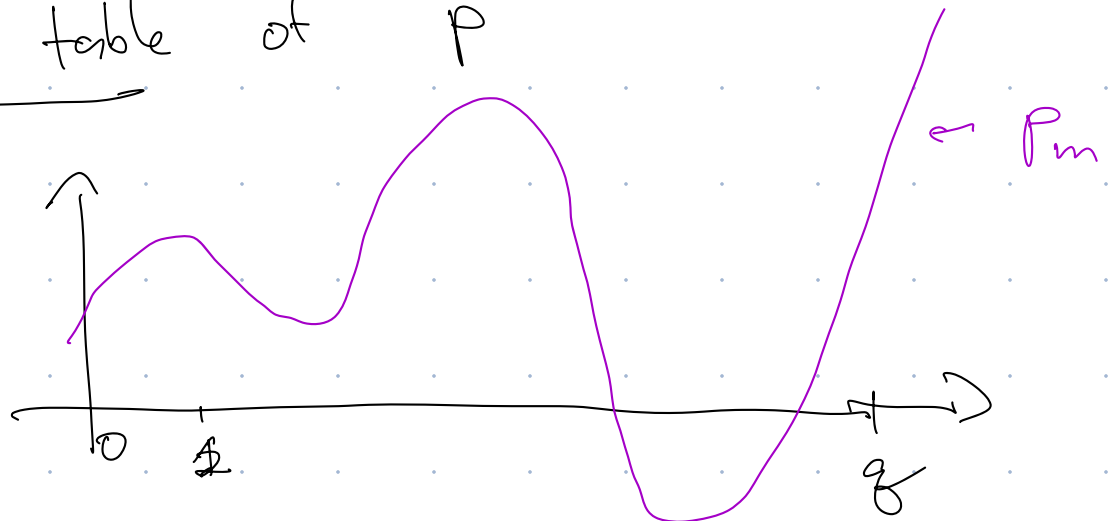
Given a message  $m \in \mathbb{F}_q^k$ , encode it into a polynomial

$$p_m(x) = m_1 + m_2 x + m_3 x^2 + \dots + m_k x^{k-1}$$

degree  $k-1$

Then the encoding of  $m$  will be the evaluation table of  $p$

Enc  
 $m \rightarrow$



Fact: Any two degree- $r$  polys that agree on  $r+1$  points are actually identical!

$\Rightarrow$  If  $m \neq m'$ ,  $\text{Enc}(m) \neq \text{Enc}(m')$  differ on at least  $q-k+1$  points

So we have a

$$[q, k, q-k+1]_q$$

Code over  $\mathbb{F}_q$  (can convert to bits if needed)

Quantumly: The analog of a linear code is a stabilizer code.

In a classical linear code, the code subspace  $S$  is the null space of some

matrix  $H \in$  "parity check matrix"  
 $\begin{pmatrix} -h_1- \\ -h_e- \end{pmatrix}$

$$x \in S \Leftrightarrow \langle h_i, x \rangle = 0 \quad i \in [e]$$

In a stabilizer code, the code subspace is the common +1 eigenspace of a collection of Pauli operators, called stabilizer generators

$$|\psi\rangle \in S \Leftrightarrow \langle \psi | P_i | \psi \rangle = I \quad i \in [e]$$

Generalizes parity checks

$$h_i \rightarrow Z^{h_i}$$

Very often, can get away with taking the stabilizer generators to be either tensor products of  $X$ 's or tensor products of  $Z$ 's.

Such codes are called CSS

CSS code = a classical linear code in the  $X$  basis and  $Z$  basis

9/24/26

## Lecture 4: The Yamakawa-Zhandry Problem and the role of structure in quantum speedups

When defining query complexity, we introduced the concept of **promise problems**: only need to solve the problem on  $x \in S_{YES} \cup S_{NO}$   $\subseteq \{0,1\}^N$ .  
possibilities

All the famous examples of exponential quantum speedup are for problems where the promise set is highly structured.

- Bernstein-Vazirani
- Simon's
- Welded trees

And even in the non-oracle world, Shor's alg solves a highly structured task.

Question: Is structure necessary for quantum speedups?

A lot of interesting results around this in the query setting.

Thm: [Beals et al '98]

For any **total function**, the quantum query complexity is polynomially related to the classical deterministic query complexity.

$$D(f) = O(Q(f)^6)$$

(Shown via polynomial method!)

So some sort of promise is necessary for exponential speedup. In fact, it's conjectured that the promise set must be **exponentially small** to get an exponential speedup!

Conjecture [Aaronson-Ambainis '09]:

Let  $Q$  be a quantum alg. making  $T$  queries to a random oracle  $\mathcal{O}$ . Then  $\forall \epsilon, \delta > 0, \exists$

a classical deterministic alg.  $C$  making  $\text{poly}(T, 1/\epsilon, 1/\delta)$

queries s.t. on  $1-\delta$  fraction of oracles  $\mathcal{O}$ ,

$$C^0 \approx_{\epsilon} \Pr [Q^0 = \text{accept}]$$

In other words, any quantum alg. can be classically simulated on most of its inputs.

This conjecture is still widely believed and progress has recently been made towards solving it...

But, the AA conjecture is only for decision problems!

Yamakawa-Zhandry '22: There exists a search problem where quantum achieves an exponential speedup w.h.p. for random oracles. Moreover, this problem is in NP! (Can efficiently check soln.)

So even relatively unstructured problems admit exponential q. speed-ups!

$\chi^2$  is inspired by ideas from lattice-based cryptography, and in turn has inspired the

DQI algorithm for combinatorial optimization.

## The YZ problem:

Think  
Fqg

Let  $\Sigma$  be some big alphabet, and let  $C$  be a (linear) code over  $\Sigma^n$ .

Let  $h_i: \Sigma \rightarrow \{0, 1\}$  be a function for each  $i \in [n]$ .  
↑  
encoded in the oracle

"hash  
function"

The task is to find  $x \in C$  s.t.

$$(h_1(x_1), h_2(x_2), \dots, h_n(x_n)) = (0, 0, \dots, 0)$$

This is clearly in  $\text{FNP}$  for all values of  $h$ :  
↓  
"Search NP"

given an  $x$ , just check whether it's in  $C$ , and that each coordinate hashes to 1.

Thm: There is a quantum alg making  $\text{poly}(n)$  queries that solves this problem for a random  $h$  with very high probability. OTOH, no classical alg making subexponentially many queries can do this. (for a certain choice of code!)

## Classical hardness:

The classical hardness will use a property called "list-recoverability". Essentially, this means that few codewords are consistent with the locations in  $h$  queried by the algorithm.

Def: A code  $C \subseteq \Sigma^n$  is  $(\zeta, \ell, L)$ -list recoverable if for any subsets  $S_i \subseteq \Sigma$ ,  $|S_i| \leq \ell$  for each  $i$ .

$$\left| \left\{ (x_1, \dots, x_n) \in C : \left| \{i : x_i \in S_i\} \right| \geq (1-\zeta)n \right\} \right| \leq L$$

"At most  $L$  codewords are consistent with a  $1-\zeta$  fraction of the lists, if each list is of size at most  $\ell$ "

Code parameters: Our code will be  $(\zeta, \ell, L)$ -list rec.

$$\text{for } \zeta = \Omega(1), \ell = 2^{\Omega(n^c)}, L = 2^{\tilde{O}(n^{c'})}$$

$0 < c < 1$                        $0 < c' < 1$

Idea: Suppose a classical alg. makes less than  $\ell$  queries to  $h$ . Then there are at most  $L$  codewords consistent with the queries.

The chance that any fixed  $x \in C$  is a solution is

$$\Pr_h \left[ h_1(x_1) = 0 \wedge h_2(x_2) = 0 \wedge \dots \right] = \frac{1}{2^n}$$

So by union bound, nothing in the list is a solution.

This is just an intuitive sketch, ignoring adaptivity of queries, but "morally" this is the proof!

Quantum easiness:

This is the fun part: we will see a very clever algorithmic trick!

Dream: Observe that it's easy to prepare the following states:

$$|\psi_c\rangle = \frac{1}{\sqrt{|C|}} \sum_{x \in C} |x_1 x_2 \dots x_n\rangle$$

$$|\psi_h\rangle = \frac{1}{\sqrt{|h^{-1}(0)|^n}} \sum_{\substack{x \in \Sigma^n: \\ h_1(x_1) = 0 \\ \vdots \\ h_n(x_n) = 0}} |x_1 x_2 \dots x_n\rangle$$

We want a point in the intersection of the supports of these two states. Suppose we could compute the state

$$|\psi\rangle \propto |\psi\rangle \odot |\psi_n\rangle$$

entrywise product of coefficients.

Then measuring  $|\psi\rangle$  will give us a point in the intersection!

It will turn out that if the code has some extra properties then this multiplication is possible!

Entrywise multiplication:

Let's try going into the Fourier domain!

Recall the FT over prime finite fields

$$\mathcal{F}|x\rangle = \frac{1}{\sqrt{q}} \sum_y \omega^{x \cdot y} |y\rangle$$

$\omega$   $q$ 'th root of unity

A Fourier transform converts products into convolutions:

$$|\alpha\rangle = \sum_x \alpha_x |x\rangle$$

$$|\beta\rangle = \sum_y \beta_y |y\rangle$$

$$\mathcal{F}|\alpha\rangle = \sum_x \hat{\alpha}_x |x\rangle$$

$$\frac{1}{\sqrt{q}} \sum_z \omega^{xz} \alpha_z$$

$$\mathcal{F}|\beta\rangle = \sum_y \hat{\beta}_y |y\rangle$$

$$\frac{1}{\sqrt{q}} \sum_z \omega^{yz} \beta_z$$

$$|\alpha\rangle \odot |\beta\rangle = \sum_x \alpha_x \beta_x |x\rangle$$

$$\mathcal{F}|\alpha\rangle \odot \mathcal{F}|\beta\rangle = \frac{1}{\sqrt{q}} \sum_{xy} \omega^{xy} \alpha_x \beta_y |y\rangle$$

$$= \frac{1}{\sqrt{q^3}} \sum_y \left( \sum_{\substack{x \\ uv}} \omega^{xy - xu - xv} \hat{\alpha}_u \hat{\alpha}_v \right) |y\rangle$$

Sum over  $x$  vanishes unless  
 $y - u - v = 0 \Leftrightarrow v = y - u$

$$= \frac{1}{\sqrt{q}} \sum_y \sum_u \hat{\alpha}_u \hat{\alpha}_{y-u} |y\rangle$$

convolution

$$= \frac{1}{\sqrt{q}} \sum_y \sum_u \hat{\alpha}_u \hat{\alpha}_y |y+u\rangle$$

change of variables

normalization

Inverse

$$\alpha_x = \frac{1}{\sqrt{q}} \sum_z \omega^{-xz} \hat{\alpha}_z$$

$$|\alpha\rangle|\beta\rangle \xrightarrow{F \otimes F} |\hat{\alpha}\rangle|\hat{\beta}\rangle \\ = \sum_{u,v} \hat{\alpha}_u \hat{\beta}_v |u\rangle|v\rangle$$

coherent  
addition

$$| \xrightarrow{\quad} \sum_{u,v} \hat{\alpha}_u \hat{\beta}_v |u\rangle|v\rangle |u+v\rangle$$

uncompute

$$| \xrightarrow{\quad} \sum_{u,v} \hat{\alpha}_u \hat{\beta}_v |0\rangle|v\rangle |u+v\rangle$$

$$| \xrightarrow{???} \sum_{u,v} \hat{\alpha}_u \hat{\beta}_v |0\rangle|0\rangle |u+v\rangle$$

Need to coherently uncompute  $v$  from  $u+v$ !

Idea: suppose the states  $|\alpha\rangle, |\beta\rangle$  are structured so that  $u$  is a codeword of some code, and  $v$  is low weight (below distance).

Then  $|u+v\rangle|v\rangle \rightarrow |u+v\rangle|0\rangle$  is the inverse of the decoding unitary!

For us:

$$|\alpha\rangle = |\psi_C\rangle = \frac{1}{\sqrt{|C|}} \sum_{x \in C} |x\rangle$$

$$|\alpha\rangle = \frac{1}{\sqrt{|C|}} \cdot \frac{1}{\sqrt{q^n}} \sum_{\substack{x \in C \\ z \in \mathbb{F}^n}} \omega^{z \cdot x} |z\rangle$$

This sum vanishes unless  $z \in C^\perp$

$$= \frac{\sqrt{|C|}}{\sqrt{q^n}} \sum_{z \in C^\perp} |z\rangle = \frac{1}{\sqrt{|C^\perp|}} \sum_{z \in C^\perp} |z\rangle$$

For a linear code  $C$ ,  $C^\perp$  is called the dual code

$$|\beta\rangle = |\psi_h\rangle = \left( |h_1^{-1}(\omega)\rangle \right) \otimes \left( |h_2^{-1}(\omega)\rangle \right) \otimes \dots$$

where each

$$|h_i^{-1}(\omega)\rangle = \frac{1}{\sqrt{|h_i^{-1}(\omega)|}} \sum_{\substack{x \in \mathbb{F}^n \\ h_i(x) = \omega}} |x\rangle$$

$$|h_i^{-1}(\omega)| \approx \frac{q}{2}$$

$$|h_i^{-1}(0)\rangle \approx \sqrt{\frac{2}{g}} \sum_{x \in \Sigma} \mathbb{1}[h_i(x)=0] |x\rangle$$

$$= \sqrt{\frac{2}{g}} \sum_{x \in \Sigma} \left( \frac{1 + (-1)^{h_i(x)}}{2} \right) |x\rangle$$

$$= \frac{1}{\sqrt{2}} \left( \frac{1}{\sqrt{g}} \sum_{x \in \Sigma} |x\rangle \right) + \frac{1}{\sqrt{2}} \left( \frac{1}{\sqrt{g}} \sum_{x \in \Sigma} (-1)^{h_i(x)} |x\rangle \right)$$

$$\mathcal{F} |h_i^{-1}(0)\rangle \approx \frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} \left( \frac{1}{g} \sum_{xy} \omega^{xy h_i(x)} |y\rangle \right)$$

most coeffs. here have  
 $1.1 \approx \frac{1}{\sqrt{g}}$

$$\text{So } \mathcal{F} |\psi_h\rangle = \sum \beta_y |y\rangle$$

"most"  $y$  have about  $\frac{n}{2}$  0's  
 and  $\frac{n}{2}$  nonzero entries

So we can implement uncomputation if

$C^\perp$  can handle  $\gamma$  errors up to weight  $\approx \frac{n}{2}$ !  
 random

In the end, turns out a particular type of Reed-Solomon code satisfies the properties we want.

The YZ Algorithm more abstractly:

The task we're trying to solve is to simultaneously satisfy a linear system (easy!)  
+ a non-linear constraint (easy!)

The "entrywise multiplication" trick reduces this to solving a decoding task for the dual of the linear system.

Regev '05: Want to find  $x \in \ker(A)$  over  $\mathbb{Z}_q^n$  (SIS)  
st.  $\forall i, |x_i| \leq \text{small}$

Reduce to decoding: given  $y = \underbrace{u}_{\in \ker(A)} + \underbrace{e}_{\text{low weight}}$   
find  $e$  (LWE)

DQI (Jordán et al '24): Want to find  $x \in C$  maximizing  
 $\sum_{i=1}^n f_i(x_i)$   $\leftarrow$  constraint